

Разработка механизма кэширования постраничного индекса LSM-дерева на основе B-деревьев



студент: Антон Кузнец
руководитель: Константин Осипов

LSM

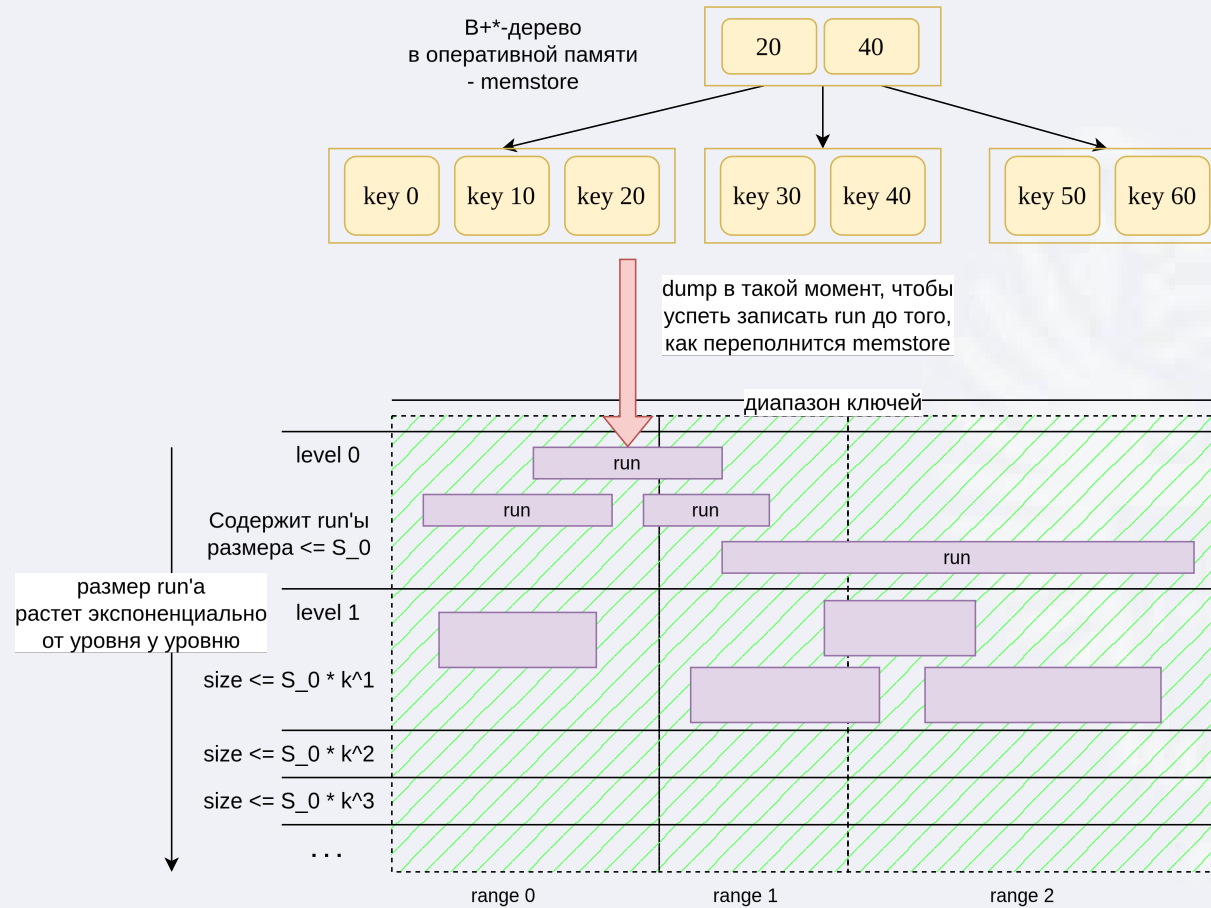
LSM-tree как идея был предложен в середине 1990-х.

Опубликован в статье *Patrick O'Neil, Edward Cheng, Dieter Gawlick, Elizabeth O'Neil, «The Log-Structured Merge-Tree (LSM-Tree)», 1996, Acta Informatica.*

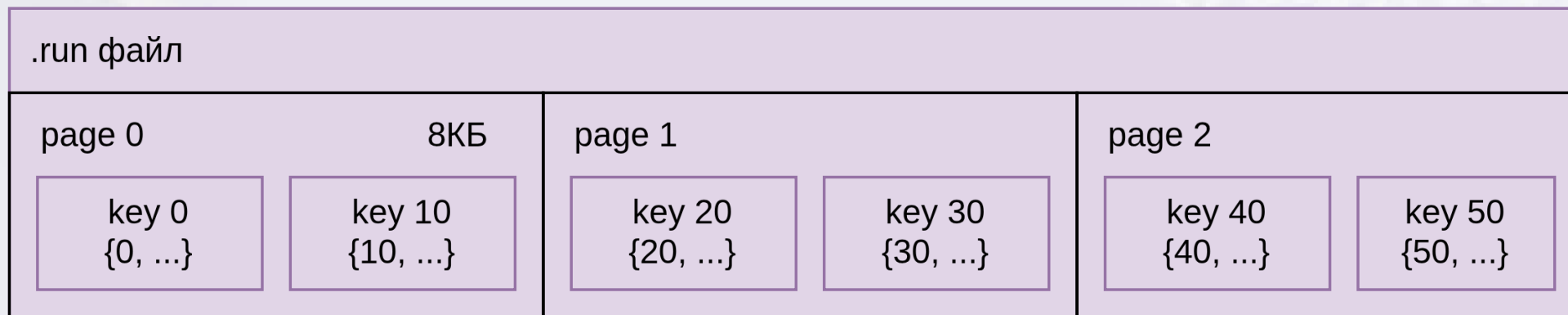
Первое широкое промышленное распространение LSM-подход получил позже, в системах типа Google Bigtable (2000-е), а затем HBase, Cassandra, LevelDB, RocksDB и т.д.



LSM (Vinyl)



LSM (Vinyl) .run файл



кортежи отсортированы по
возрастанию ключа

LSM

Зачем нужен compaction:

- новые записи дешево писать в память и периодически сбрасывать в новые маленькие гуп'ы
- но если просто накапливать много гуп'ов, чтение станет дорогим, потому что придётся проверять слишком много гуп'ов

Поэтому LSM, и в частности Vinyl, балансирует между тем чтобы:

- сделать запись дешевой
- не дать чтению деградировать из-за большого числа гуп'ов

Для этого периодически несколько старых гуп'ов сливаются в один новый, чтобы уменьшить read amplification.

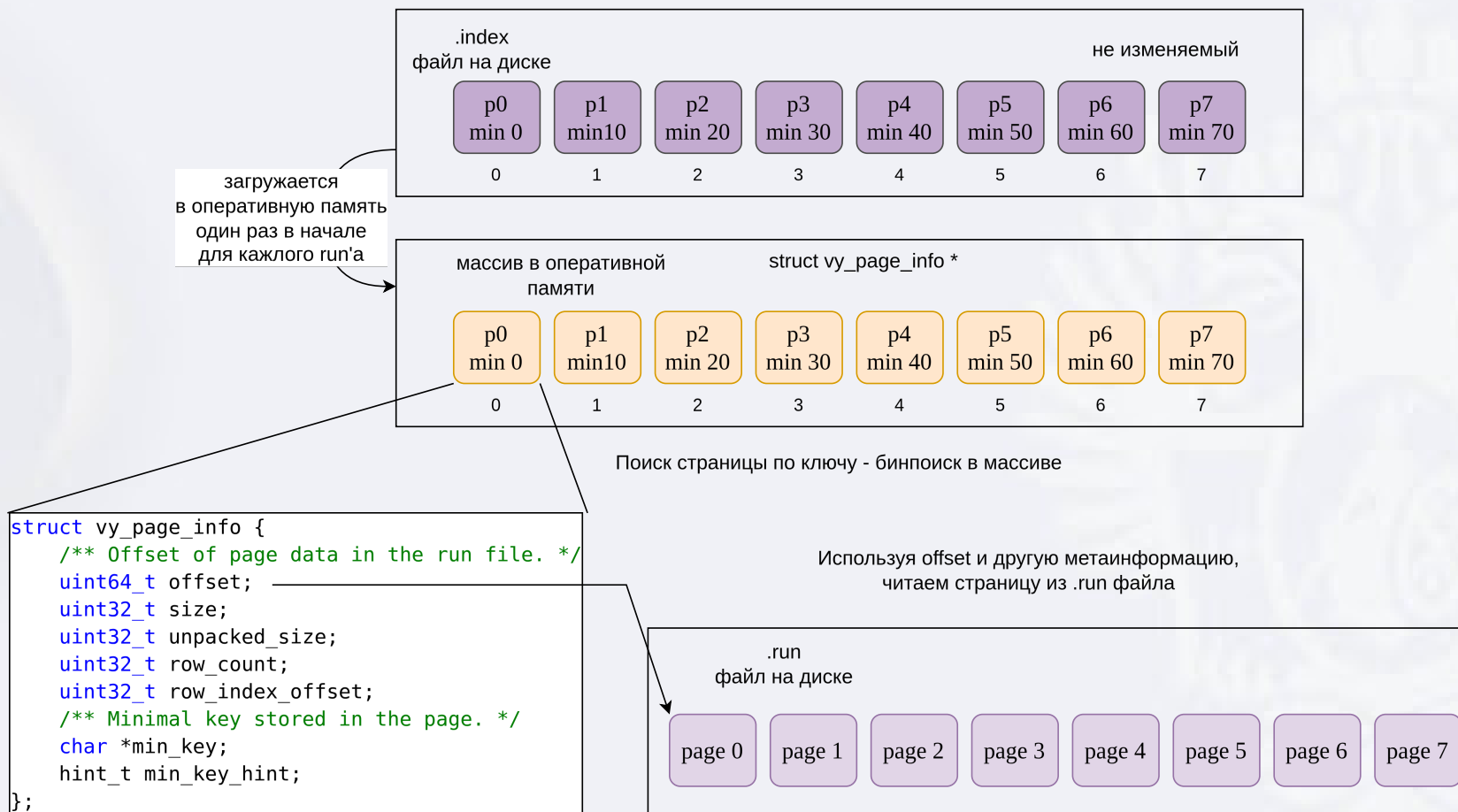




Стратегия compaction в LSM балансирует между несколькими свойствами:

- Write amplification
Чем более агрессивно происходит compaction, тем больше одни и те же данные перезаписываются. Было бы плохо переписывать большие объёмы данных слишком часто.
- Read amplification
Если не compact-ить совсем, то становится много слоёв из данных runs, в том числе уже не актуальных, и чтению приходится проверять много источников.
- Space amplification
Пока старые версии ключей и tombstones не схлопнуты, на диске лежит много «мёртвых» данных.
- предсказуемость фоновой нагрузки

Page index в оперативной памяти



Page index в оперативной памяти



page_size в Vinyl по умолчанию 8 КБ.

Одна struct vy_page_info без учёта min_key занимает примерно 40 Б.

$40 / 8192 = 0.0048$ (0.5% от объёма данных)

Если добавить, что у каждой страницы ещё отдельно хранится min_key по указателю, то реалистичнее считать не 40 Б, а примерно 56–72 Б на страницу для коротких/средних ключей. Тогда overhead получается уже порядка 0.68%–0.88%, а с длинными ключами может уйти и к 1%+.

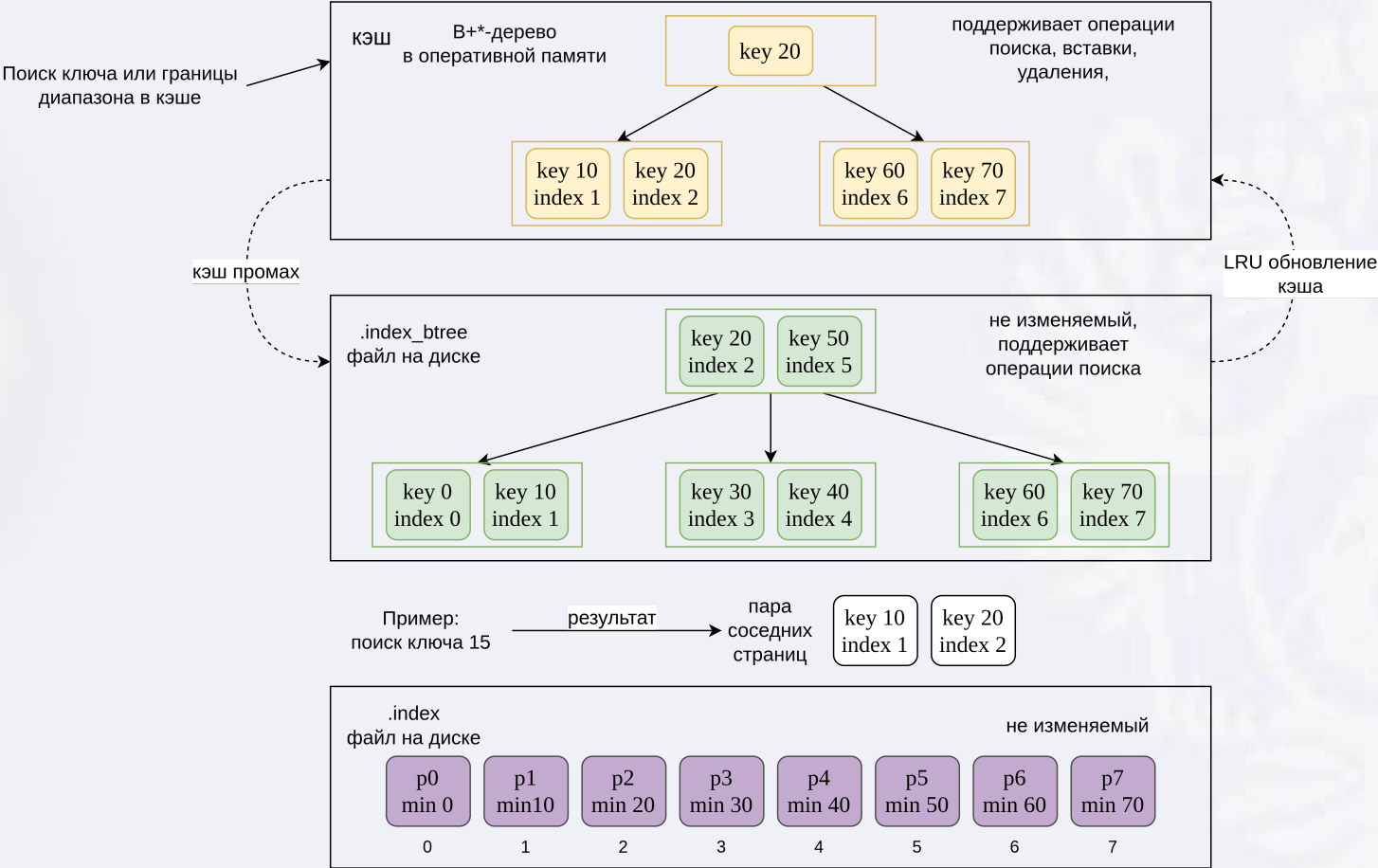
Проблема: Для терабайтного набора данных это сотни мегабайт – гигабайты одних только метаданных в оперативной памяти.

Задачи

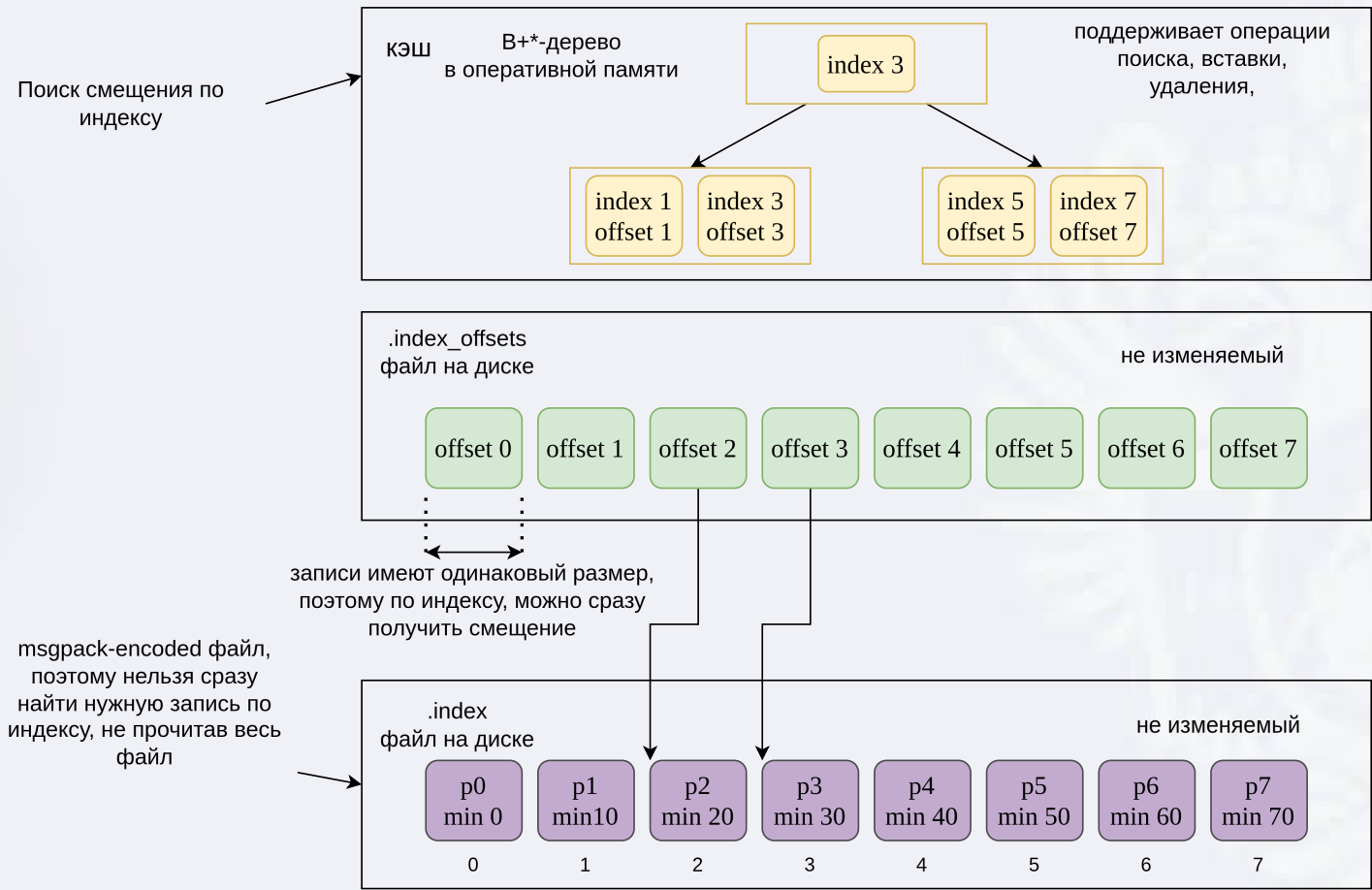


- проанализировать устройство LSM-деревьев и особенности реализации Vinyl в Tarantool
- исследовать существующий постраничный индекс Vinyl и определить ограничения его масштабирования по памяти
- сформулировать требования к новому механизму постраничного индекса
- разработать структуру постраничного индекса с размещением основной части метаданных на диске и ограниченным рабочим набором в памяти
- реализовать предложенный подход в кодовой базе Tarantool
- сравнить разработанное решение с in-memory подходом
- провести экспериментальную оценку по памяти и производительности

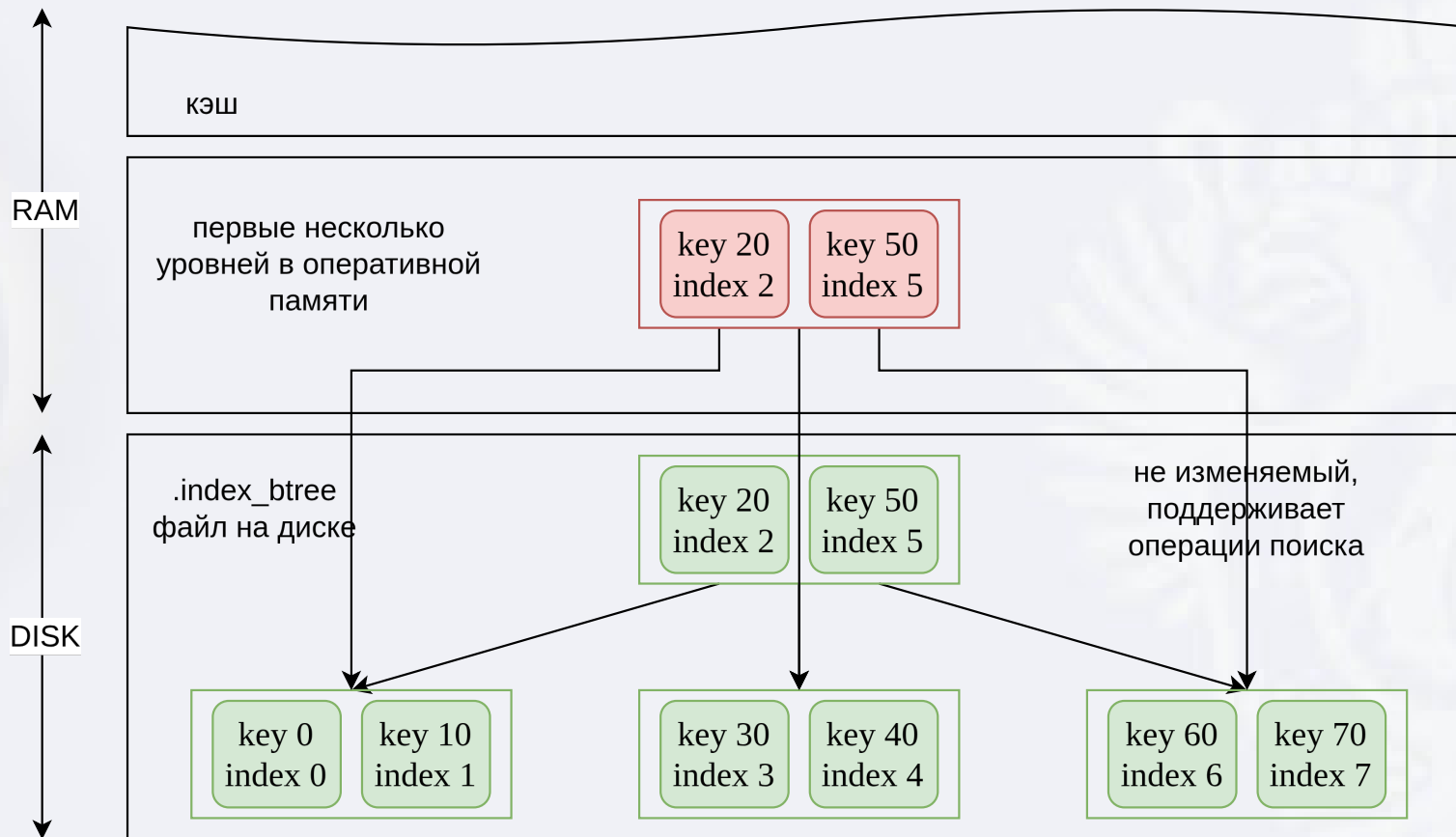
Индекс страницы по ключу



Страница по индексу



В-дерево



Бенчмаркинг: методика



YCSB-подобные нагрузки Vinyl: workload A, B и C (pico_ycsb, набор 200 МБ).

Сравнивались две реализации постраничного индекса:

- old — метаданные страницы в RAM
- new — B-дерево на диске, в RAM только кэши (соотношение ключи : vy_page_info = 1:4)

Квота кэшей варьировалась от 0 до 2 МБ; резидентная верхушка B-дерева отключена (d0). Перед прогонами сбрасывались дисковые кэши ОС, vinyl_cache = 0.

Бенчмаркинг: метрики



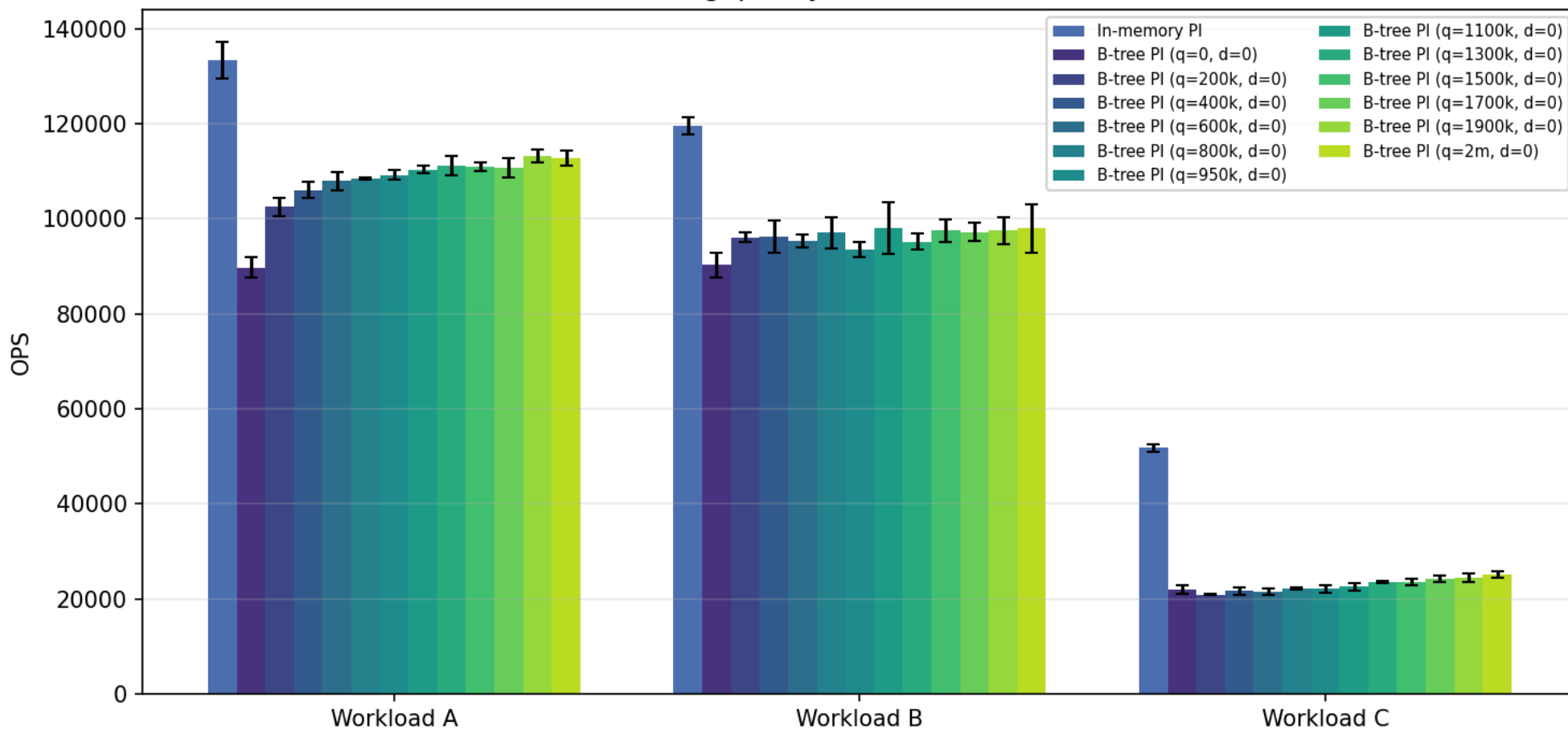
- throughput (OPS) — операций в секунду
- memory.page_index — память структур постраничного индекса
- read / write / space amplification
- для new: hit rate и вытеснения по двум кэшам

Варианты новой реализации: q^*_d0 — квота кэша q^* , нулевая резидентная верхушка.

Пропускная способность

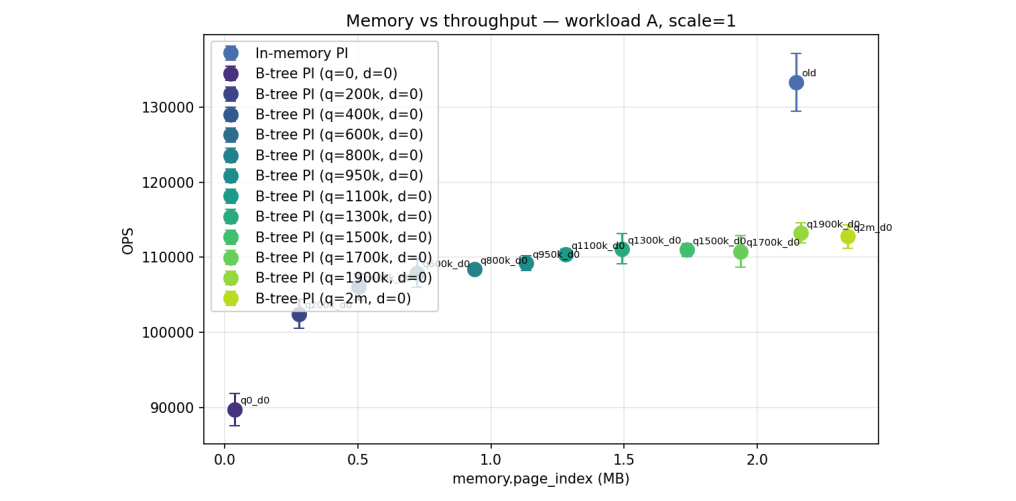


Throughput by workload (scale=1)

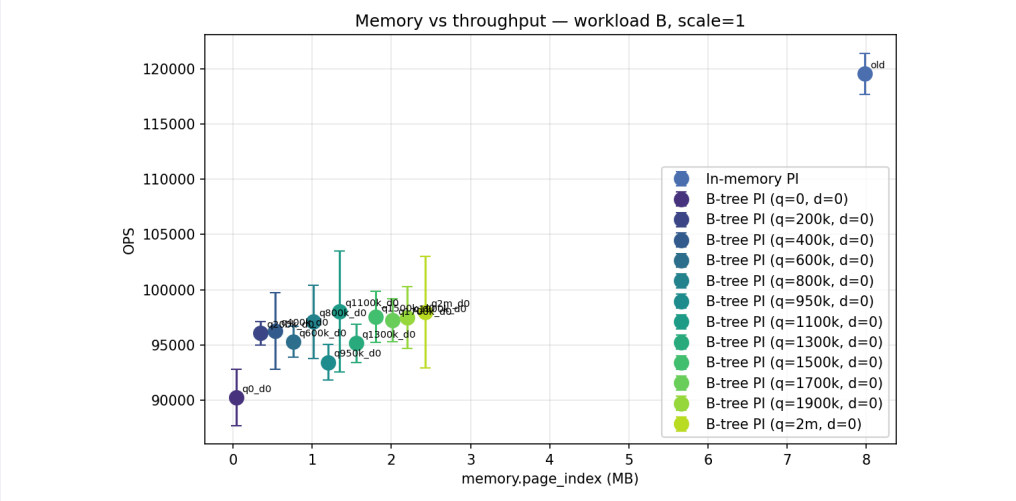


При нулевых квотах (q0_d0) throughput ниже, чем у old: A – 33%, B – 25%, C – 58%. С ростом квоты новая реализация приближается к old и выходит на плато.

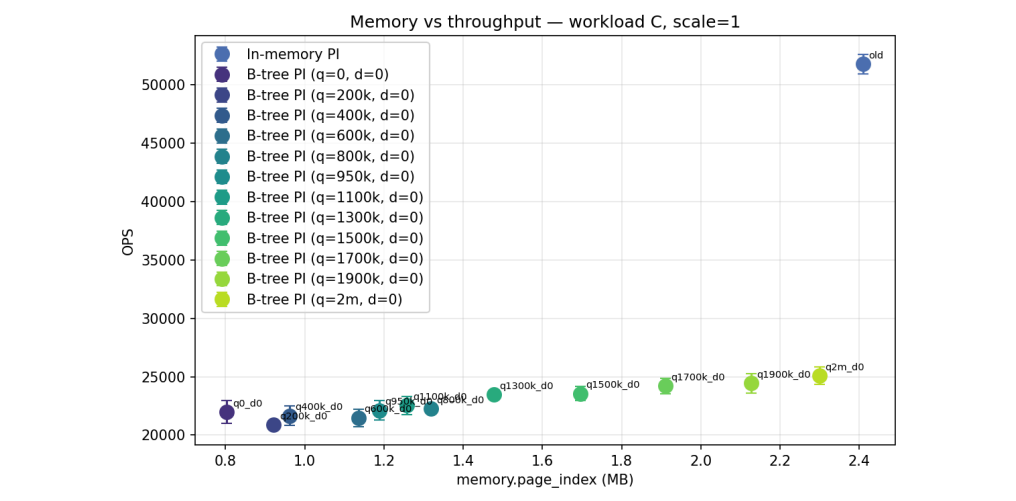
Память и пропускная способность



Workload A



Workload B



Workload C

Просадка throughput при меньшей памяти



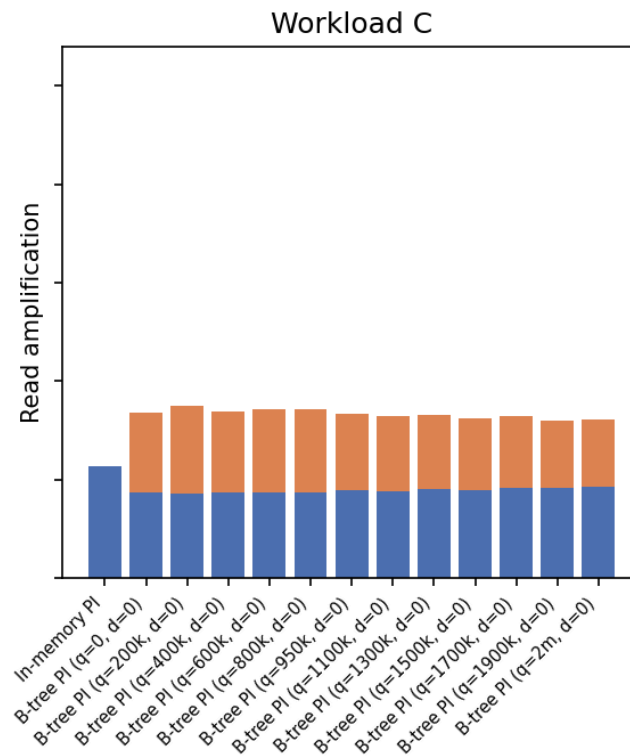
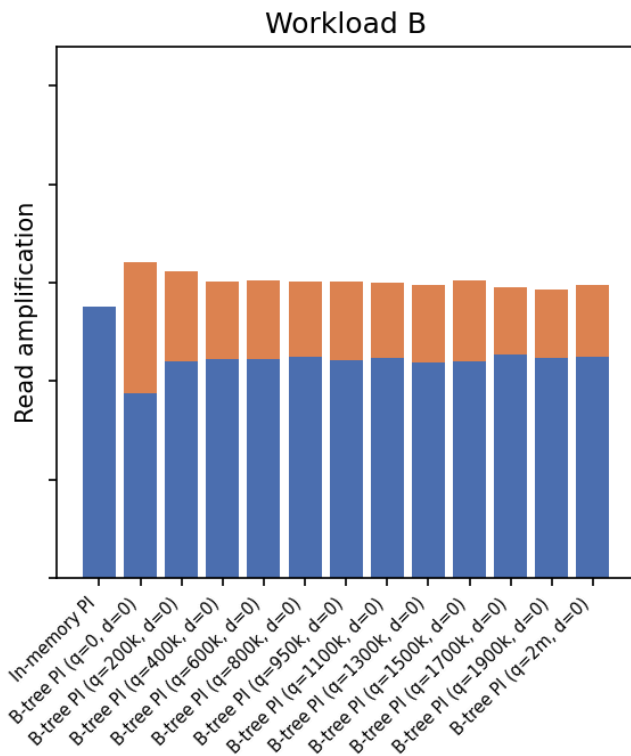
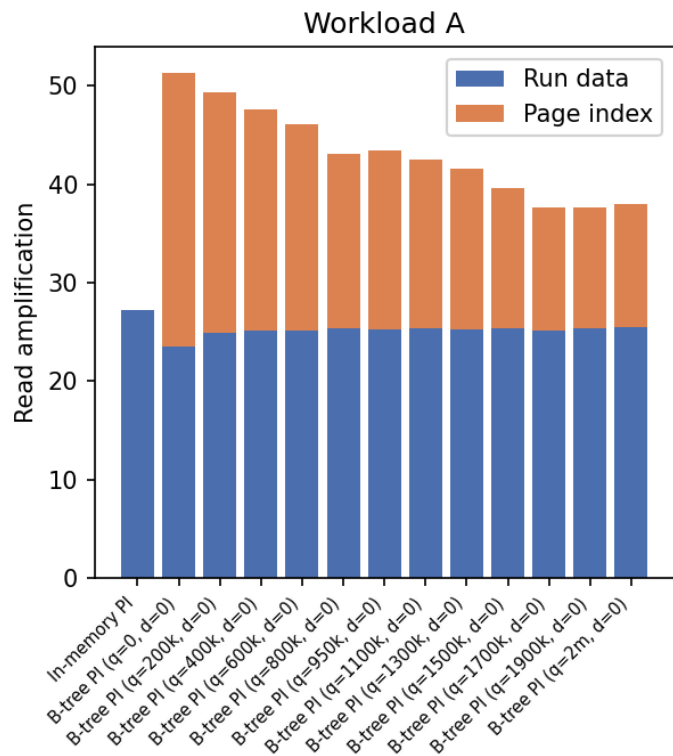
Память относительно old	Workload A	Workload B	Workload C
в 4 раза меньше	−20%	−19%	−58%
в 3 раза меньше	−19%	−18%	−58%
в 2 раза меньше	−18%	−18%	−57%

В new память ограничена квотой кэшей; при малых квотах — дополнительный I/O. При росте квоты throughput растёт до плато.

Read amplification



Read amplification decomposition (scale=1)

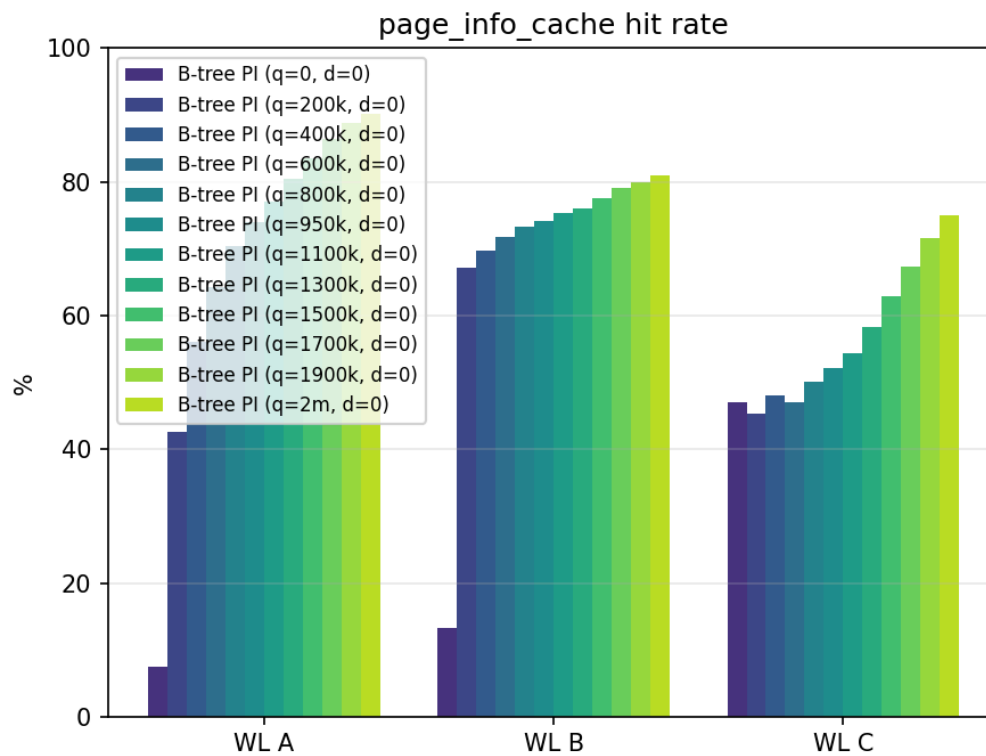
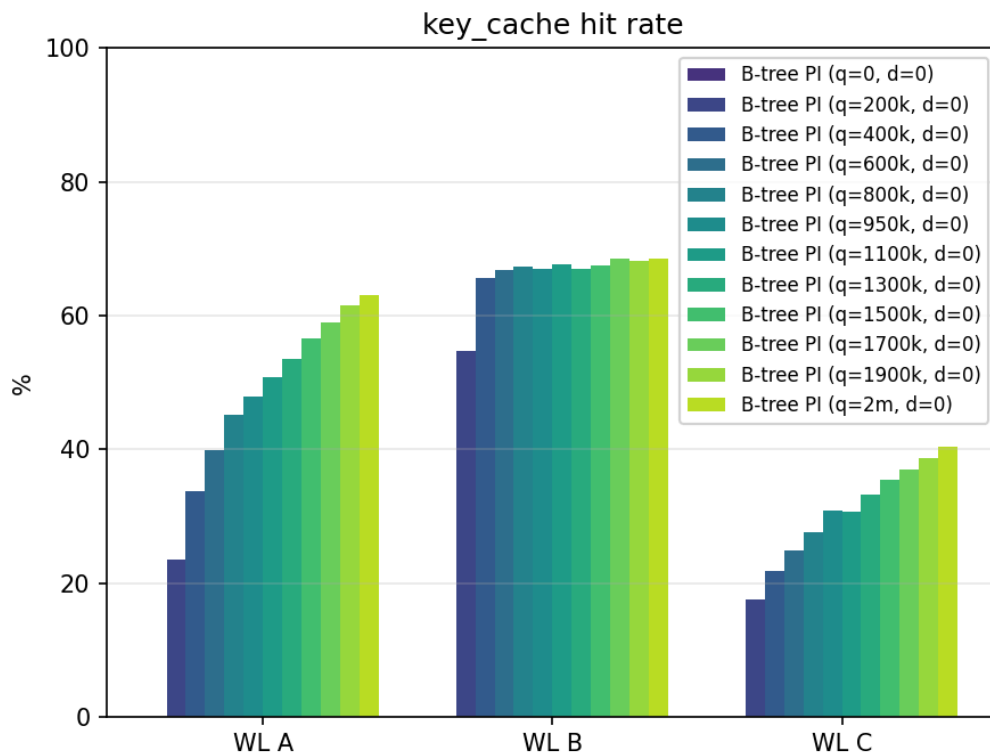


Декомпозиция: чтение данных гуп и обращения к постраничному индексу. При нулевой квоте вклад индекса максимален, с ростом квоты сжимается.

Hit rate кэшей

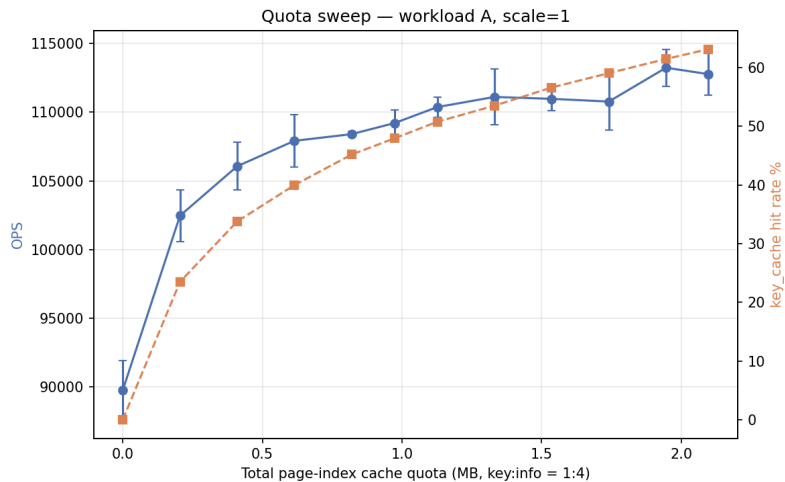


Cache hit rates (scale=1)



Рост hit rate согласуется с ростом throughput: чем больше рабочий набор в кэшах, тем реже промахи на критическом пути.

Сравнение по workload (quota sweep)



Workload A



Workload B



Workload C

Вывод



Новая схема постраничного индекса ограничивает обязательное потребление памяти за счёт хранения метаданных на диске и кэширования рабочего набора.

Появляется дополнительный компонент read amplification — чтение блоков `vy_page_info` — который заметно влияет на throughput при малых квотах.

Выбор квоты кэшей — ключевой параметр баланса между памятью и производительностью.